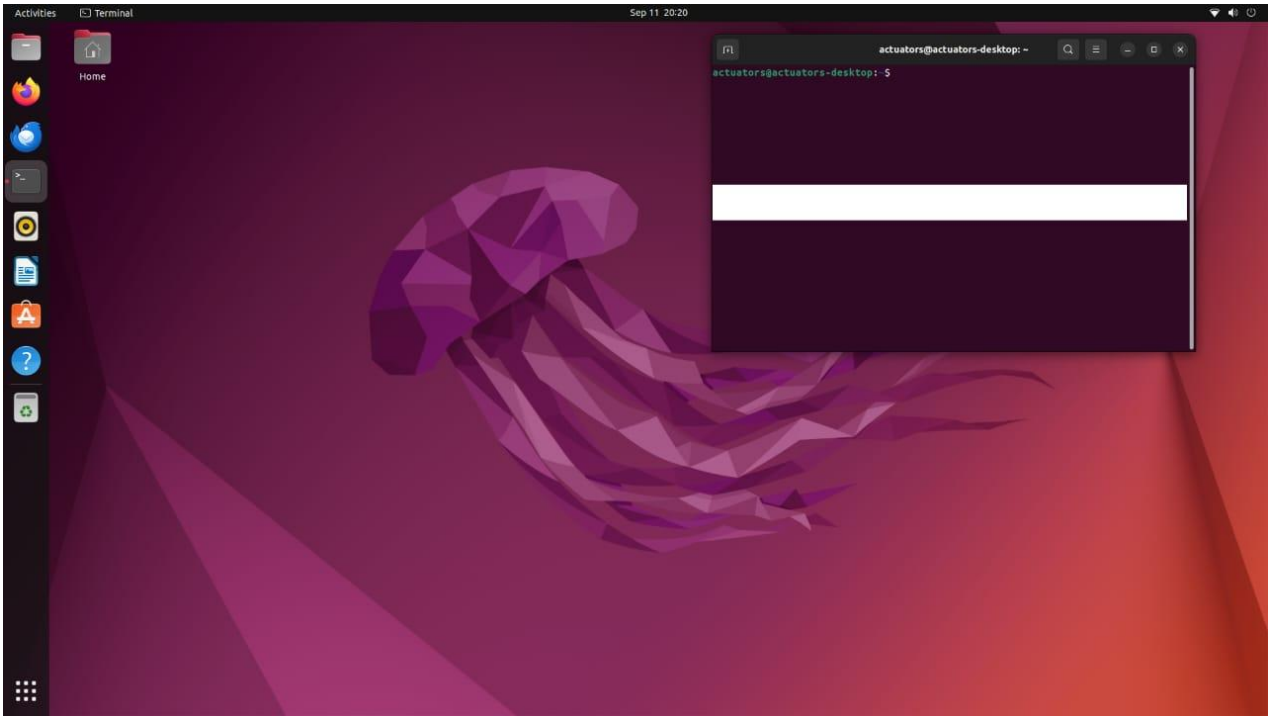
Gazebo Simulation

1. ROS2 Installation on Raspberry Pi

We began our development by installing ROS2 on Ubuntu, which was running on a Raspberry Pi with a 16GB SD card. The Raspberry Pi was connected to a monitor via HDMI and equipped with standard I/O peripherals (keyboard and mouse) to interact with the Ubuntu interface.

Following the guidance provided by Mr. Mukesh and using the official documentation, we successfully installed ROS2.

Reference: https://docs.ros.org/en/dashing/Installation/Ubuntu-Install-Debians.html



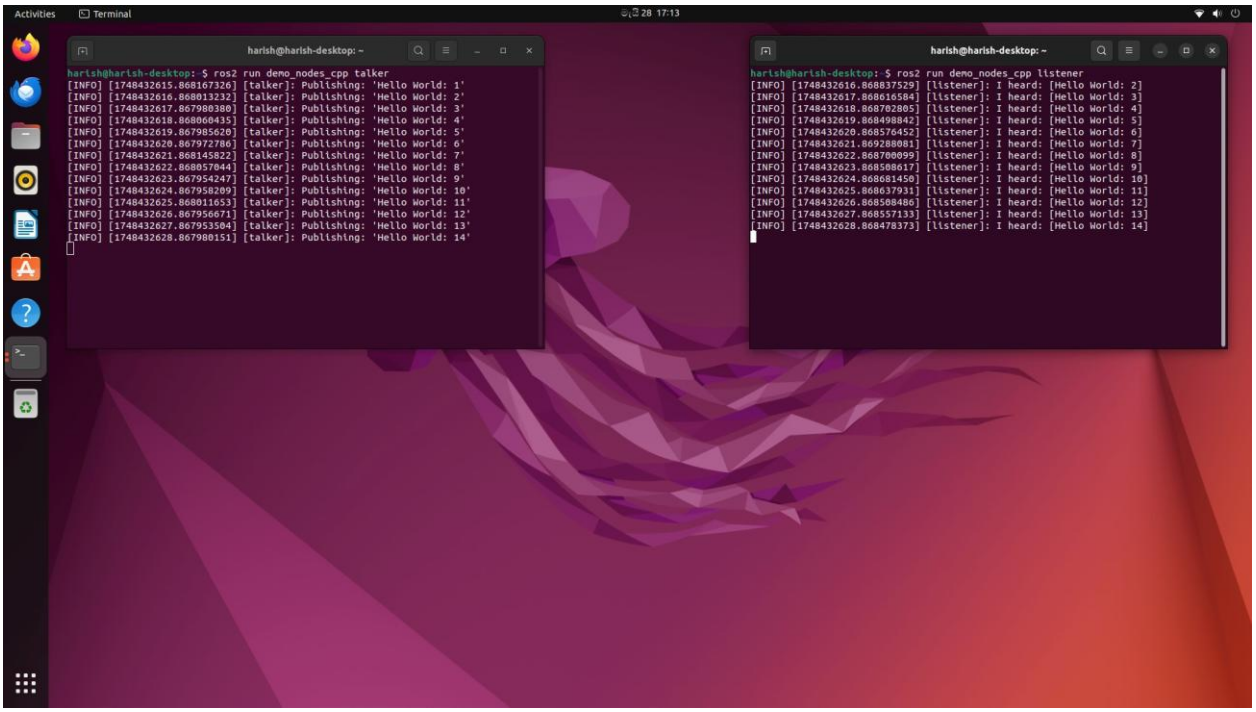
2. Verifying ROS2 Installation

To verify that ROS2 was installed correctly, we used the following commands in two separate terminals:

bash CopyEdit
ros2 run demo_nodes_cpp talker ros2 run
demo_nodes_py listener

The “talker” node publishes messages, while the “listener” node subscribes to them. This verified that ROS2's publish–subscribe mechanism was functioning as expected.

Note : Due to the unavailability of the MicroSD in our team, we joined hands with two of the members of the “Cardboard Drone” Team starting from this step.



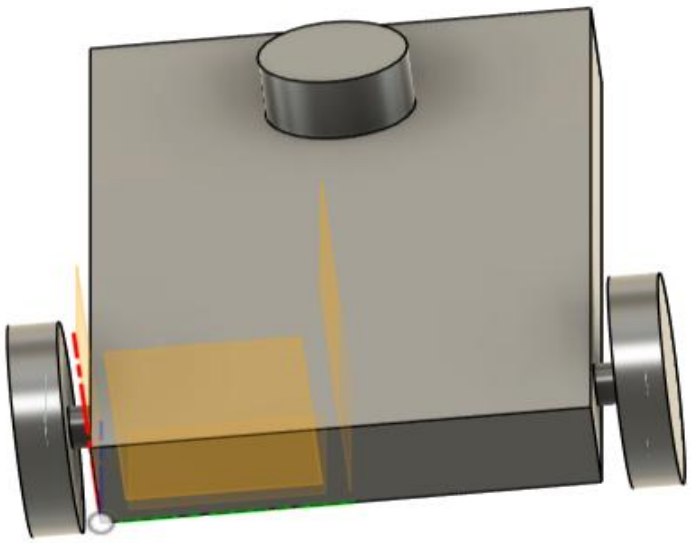
3. Designing the Robot in Fusion 360

We designed the robot model using Fusion 360. However, Fusion 360 does not natively support exporting files in URDF format. To solve this, we downloaded a URDF exporter script from a GitHub repository and integrated it into Fusion 360 via the “Scripts and Add-ins” feature.

Following this integration, we successfully exported our design in a format compatible with ROS.

Reference video: Fusion 360 to URDF | Simple Diff Drive Robot Modelling | ROS | Mappin…

Design:https://drive.google.com/drive/folders/1YRwYymGaz_z6zvXFzdWMAjwJrOvZUykm?usp=drive_link



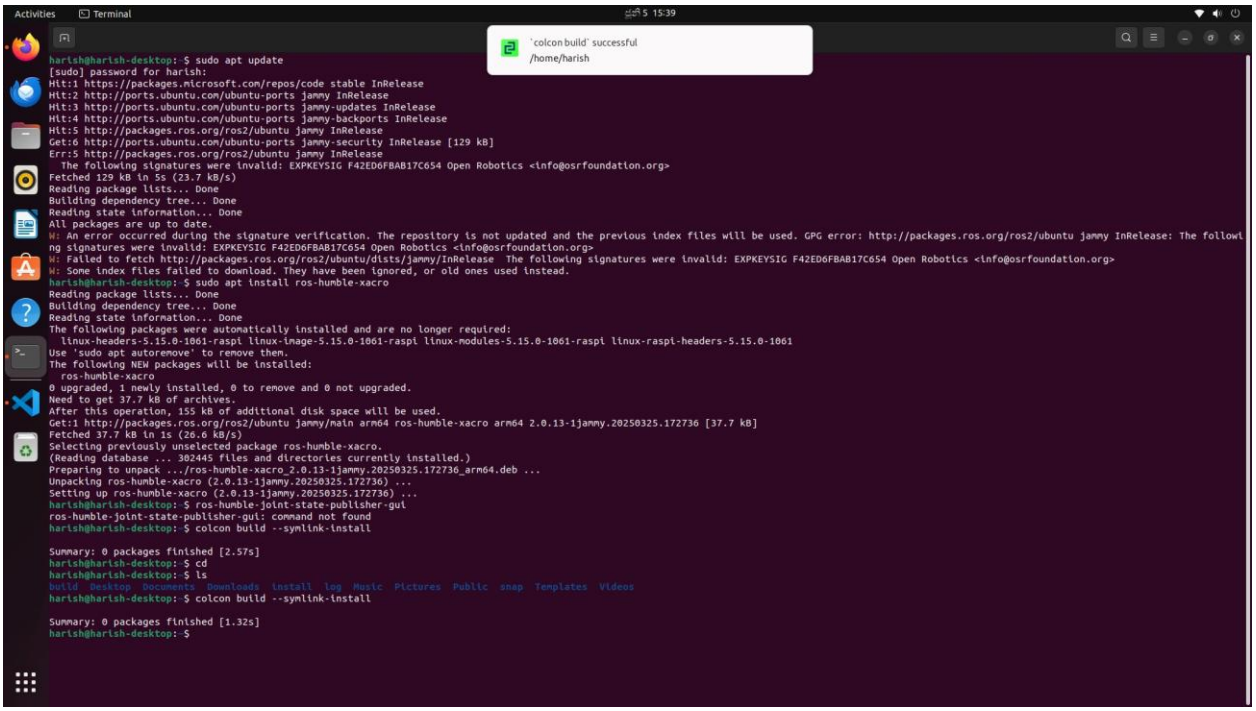
4. Installing COLCON

COLCON is a command-line build tool for ROS2 that replaces catkin in ROS1. It enables us to build packages, manage dependencies, and handle complex ROS2 workspaces.

We installed it using the following commands:

bash CopyEdit sudo
apt update
sudo apt install python3-colcon-common-extensions

Reference: https://colcon.readthedocs.io/en/released/user/installation.html



5. Installing Visual Studio Code

We chose Visual Studio Code (VS Code) as our development environment for editing .xacro and .urdf files.

Installation steps:

1. Download the .deb package from the official VS Code website.

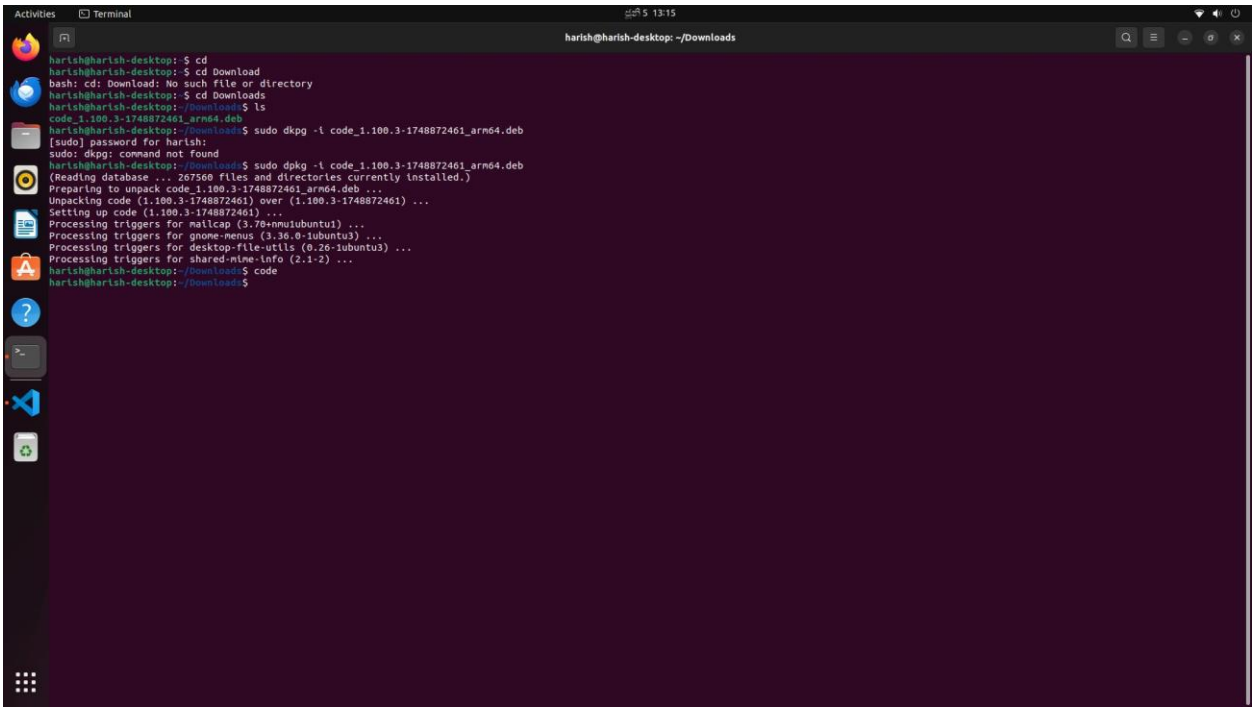
Run:

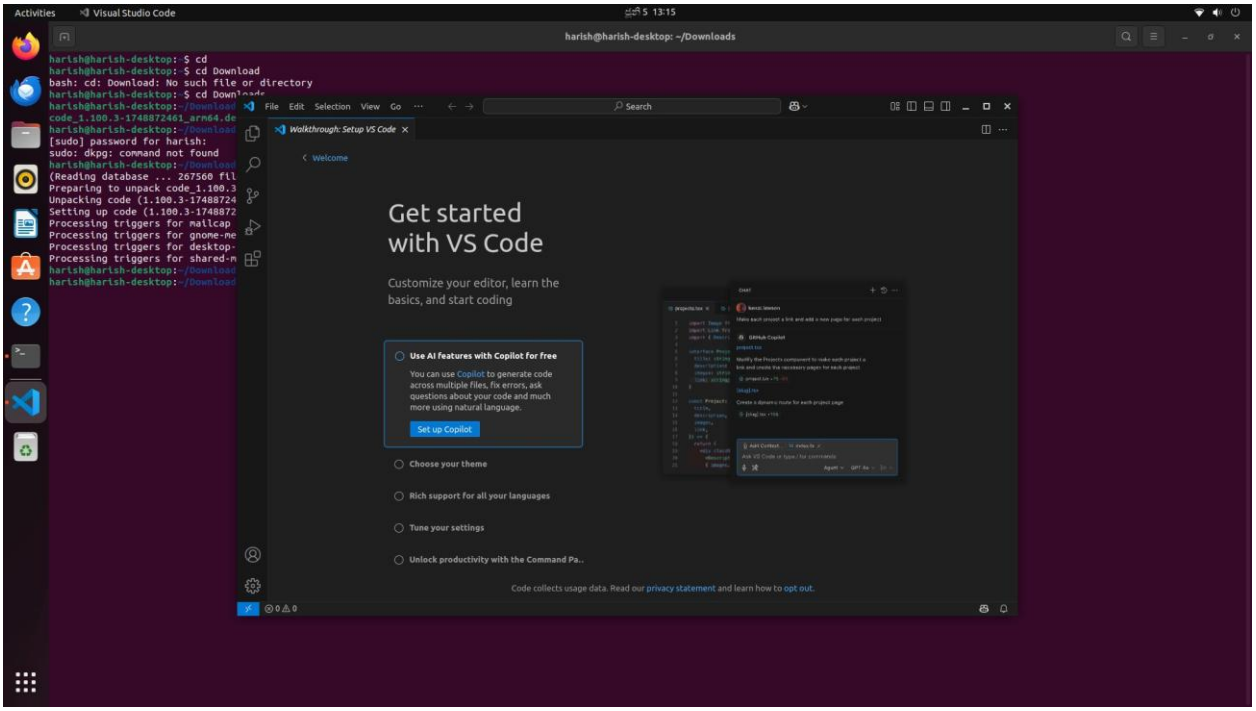
bash CopyEdit sudo
apt update
sudo dpkg -i <package_name>.deb

2.

Launch VS Code by entering:

bash
CopyEdit code





6. Creating and Sourcing a ROS2 Workspace

We created a workspace to organize our ROS2 packages and ensure environment variables are properly loaded.

Steps:

bash CopyEdit
mkdir -p ~/ros2_ws/src cd ~/ros2_ws

We then added the following line to .bashrc to automatically source the ROS2 environment whenever we open a terminal:

bash CopyEdit
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc source ~/.bashrc

This prevents us from having to manually source ROS2 in every new terminal session.

7. Installing Xacro and Joint State Publisher GUI

We installed Xacro to create modular, readable URDF files using macros. Xacro allows us to build maintainable and scalable robot descriptions.

We also installed Joint State Publisher GUI, which allows us to simulate and visualize joint states. This tool is critical during early development and testing phases, especially when hardware is not yet available.

Installation commands:

bash CopyEdit
sudo apt update
sudo apt install ros-humble-xacro
sudo apt install ros-humble-joint-state-publisher-gui

After installation, we built our workspace using:

bash CopyEdit

colcon build --symlink-install

We used the --symlink-install flag to avoid repeated rebuilds when modifying existing files. This is useful during frequent iterations. However, if we added any new files, we ran:

bash CopyEdit
colcon build

This command ensured the new files were properly registered in the build system.

To reflect changes in the robot description, we restarted the robot_state_publisher after each modification.

Reference: https://articulatedrobotics.xyz/tutorials/ready-for-ros/ros-overview

8. Error Encountered During colcon Build

After the installation and workspace setup, we attempted to launch the robot model using our Xacro-based description. However, we encountered a build error. Upon investigation, we found that the URDF/Xacro files were structured according to ROS1 conventions and were not compatible with ROS2.

Due to this error, we paused further steps such as testing robot_state_publisher, and began restructuring the files for ROS2 compatibility.

We tried few steps to resolve this problem. We use the following code to covert ROS1 to ROS2.

Step 1: Verify the URDF/Xacro file

URDF files themselves usually **don't need any changes** unless:

- You're using deprecated tags.
- You're using ROS 1-specific plugins (like Gazebo or RViz-related tags).
- Your .xacro files use old syntax (update to newer <xacro:...> format if necessary).

Test the xacro file:

bash

CopyEdit

ros2 run xacro xacro your_robot.urdf.xacro -o your_robot.urdf

Step 2: Convert launch files

ROS 2 uses **Python-based launch files** instead of XML (.launch.py instead of .launch).

Here's how to convert:

ROS 1 (XML style) xml

CopyEdit

```
<launch>

  <param name="robot_description" command="$(find xacro)/xacro $(find
your_package)/urdf/your_robot.urdf.xacro" />

  <node name="robot_state_publisher" pkg="robot_state_publisher" type="robot_state_publisher" />

</launch>
```

ROS 2 (Python style) python

CopyEdit

```
from launch import LaunchDescription from launch_ros.actions import Node from

ament_index_python.packages import get_package_share_directory import os


def generate_launch_description():

    urdf_file = os.path.join(         get_package_share_directory('your_package'),

        'urdf',

        'your_robot.urdf.xacro'

    )      return LaunchDescription([         Node(             package='robot_state_publisher',

executable='robot_state_publisher',             name='robot_state_publisher',             output='screen',

parameters=[{'robot_description': open(urdf_file).read()}]

        ),

    ])
```

Step 3: Handle dependencies

Update your package.xml and CMakeLists.txt to use ROS 2 dependencies:

In package.xml:

xml

CopyEdit

<exec_depend>robot_state_publisher</exec_depend> <exec_depend>xacro</exec_depend>

In CMakeLists.txt:

Make sure to find dependencies with:

cmake

CopyEdit

find_package(ament_cmake REQUIRED) find_package(xacro REQUIRED)

find_package(robot_state_publisher REQUIRED)

Step 4: Build and Test

1. Build your ROS 2 package:

bash CopyEdit

colcon build

2. Source your workspace:

bash CopyEdit

source install/setup.bash

3. Launch:

bash CopyEdit

```
ros2 launch your_package your_launch_file.launch.py
```

But it wont works and the error continues.